PRÁCTICA 2
Concurrencia, señales y semáforos

Fernando Blanco Varela (*fernando.blancov@estudiante.uam.es*)

Danyyil Shykerynets (*danyyil.shykerynets@estudiante.uam.es*)

Grupo 2271

30-04-2026

Escuela
Politécnica Superior

1. INTRODUCCIÓN

En esta segunda práctica, profundizamos en la programación concurrente, coordinando múltiples procesos. Introducimos el uso de señales para comunicar procesos y controlar el flujo de ejecución; y semáforos para proteger secciones críticas. Así, ampliamos el sistema hacia una red de procesos colaborativos, conviertiendo el proyecto en un sistema distribuido.

2. CUMPLIMIENTO DE REQUISITOS

Hemos cumplido todos los requisitos que se han pedido:

- Ejecución del sistema con los nuevos parámetros, duración en segundos y número de hilos.
- Múltiples procesos "Minero": Existen múltiples procesos minero que compiten por encontrar la solución. Cada minero se lanza independientemente, pudiendo lanzarse desde terminales distintas. Estos mineros tienen un tiempo de vida que determina cuánto van a estar trabajando.
- Sistema de Votaciones: Los distintos procesos mineros se comunican a través del uso de señales. Cuando un minero encuentra la solución, antes de empezar la siguiente ronda, el resto debe validarla mediante el sistema de votación.
- Protección de zonas críticas: El sistema es robusto y garantiza, con bloqueo de señales y esperas no activas, evitando la pérdida de señales a través de sigsuspend.
- Pruebas exhaustivas y control de errores.
- Análisis del rendimiento.
- Documentación con Doxygen.

3. PRUEBAS REALIZADAS

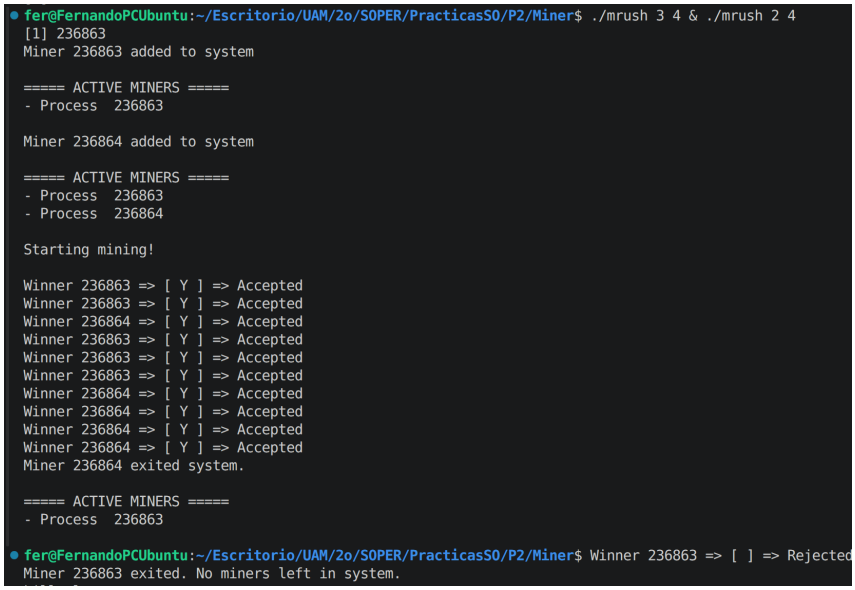
3.1. Pruebas de funcionamiento básicas

En primer lugar, verificamos que la ejecución básica del sistema reproduce un comportamiento coherente con el descrito en el enunciado de la práctica. Para ello, lanzamos varias instancias del programa `mrush` con distintos tiempos de vida y número de hilos, comprobando que:

- Los mineros se añaden correctamente al sistema.
- El primer minero inicializa los ficheros compartidos.
- Los mineros duran el tiempo indicado por entrada.
- Se arrancan rondas únicamente cuando hay al menos dos mineros activos.
- Existe un único proceso ganador por ronda.
- El resto de mineros pasan a la fase de votación.
- El registrador genera un fichero `.log` con la estructura esperada.
- Las soluciones incorrectas son rechazadas por el sistema.

De esta forma validamos que la arquitectura general del sistema es correcta antes de proceder a pruebas más específicas de concurrencia, sincronización y rendimiento.

3.1.1. Adición correcta de mineros al sistema



```
● fer@FernandoPCUbuntu:~/Escritorio/UAM/2o/SOPER/PracticasSO/P2/Miner$ ./mrush 3 4 & ./mrush 2 4
  [1] 236863
  Miner 236863 added to system

  ===== ACTIVE MINERS =====
  - Process  236863

  Miner 236864 added to system

  ===== ACTIVE MINERS =====
  - Process  236863
  - Process  236864

  Starting mining!

  Winner 236863 => [ Y ] => Accepted
  Winner 236863 => [ Y ] => Accepted
  Winner 236864 => [ Y ] => Accepted
  Winner 236863 => [ Y ] => Accepted
  Winner 236863 => [ Y ] => Accepted
  Winner 236863 => [ Y ] => Accepted
  Winner 236864 => [ Y ] => Accepted
  Winner 236864 => [ Y ] => Accepted
  Winner 236864 => [ Y ] => Accepted
  Winner 236864 => [ Y ] => Accepted
  Miner 236864 exited system.

  ===== ACTIVE MINERS =====
  - Process  236863

● fer@FernandoPCUbuntu:~/Escritorio/UAM/2o/SOPER/PracticasSO/P2/Miner$ Winner 236863 => [ ] => Rejected
  Miner 236863 exited. No miners left in system.
```

Figura 1: Ejemplo de ejecución básica del sistema de mineros

Como se puede ver, los mineros se añaden correctamente al sistema, estos mineros sólo empiezan a minar cuando hay al menos dos, y existe un único ganador, cuya solución es validada por el resto de mineros ([Y] = "Yes").

3.1.2. Inicialización de ficheros compartidos por el primer minero

Al iniciar un solo minero, el programa queda en ”stall” o en espera de la adición de un segundo. Aprovechando esto, podemos verificar que el primero cree los ficheros compartidos:

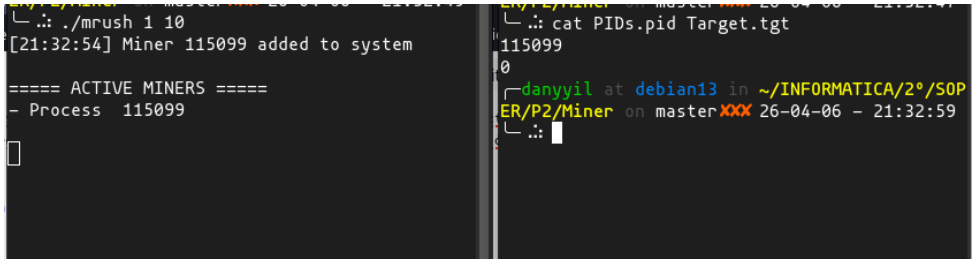


Figura 2: Correcta creación de los ficheros PID y Target

Además, verificamos que al unir un minero nuevo, el fichero de PIDs se actualiza:

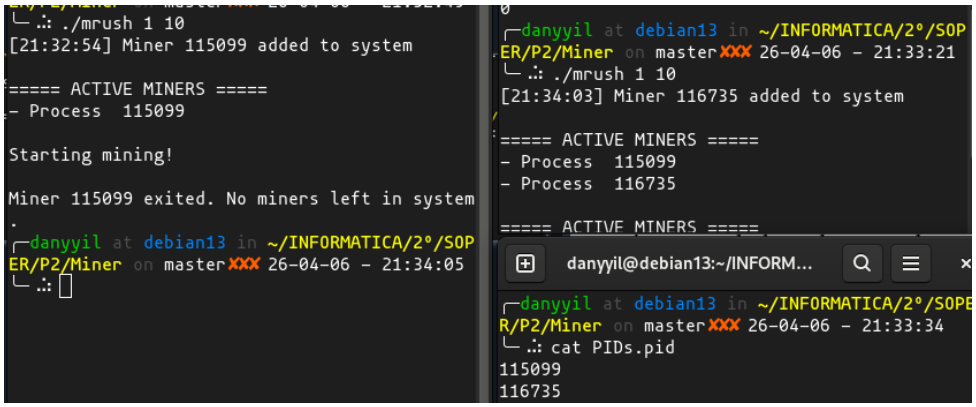


Figura 3: Adición del PID al fichero

En la terminal de abajo derecha vemos como queda el fichero de PIDs tras la adición del segundo minero.

3.1.3. Duración correcta de los mineros

Para verificar la correcta duración de los mineros en el sistema, empleamos la función time proporcionada por la shell. En este caso, usamos la función del la shell "zsh", obteniendo el siguiente resultado para 4 distintos mineros:

```
└─ .:: time ./mrush 5 5 & time ./mrush 7 5 & time ./mrush 2 10 & time ./mrush 10 1
[1] 110861
[2] 110862
[3] 110865
[21:16:30] Miner 110867 added to system

===== ACTIVE MINERS =====
- Process  110867

[21:16:30] Miner 110866 added to system
Starting mining!


===== ACTIVE MINERS =====
- Process  110867
- Process  110866

[21:16:30] Miner 110864 added to system

===== ACTIVE MINERS =====
- Process  110867
- Process  110866
- Process  110864

[21:16:30] Miner 110863 added to system

===== ACTIVE MINERS =====
- Process  110867
- Process  110866
- Process  110864
- Process  110863
```

```
Miner 110866 exited system.

===== ACTIVE MINERS =====
- Process  110867
- Process  110864
- Process  110863

./mrush 2 10  0.43s user 0.04s system 23% cpu 2.003 total
[3]  + 110865 done       time ./mrush 2 10
Miner 110863 exited system.

===== ACTIVE MINERS =====
- Process  110867
- Process  110864

./mrush 5 5  0.56s user 0.04s system 11% cpu 5.007 total
[1]    110861 done       time ./mrush 5 5
Miner 110864 exited system.

===== ACTIVE MINERS =====
- Process  110867

./mrush 7 5  0.83s user 0.09s system 13% cpu 7.005 total
[2]  + 110862 done       time ./mrush 7 5
Miner 110867 exited. No miners left in system.
./mrush 10 1  0.21s user 0.03s system 2% cpu 10.003 total
```

Figura 4: Creación de los procesos y adición al sistema. Duración en segundos

Como se puede observar especialmente en la imagen de la derecha, los mineros tienen una duración prácticamente exacta a la indicada por el parámetro, siendo ligeramente mayor debido al setup inicial (señales, alarma, etc.) y final (eliminación del timer, comunicación al logger, liberación de memoria...)

3.1.4. Generación de un registro. Ganador único por ronda

Podemos comprobar que se genera de manera correcta un registro (log) para cada minero, con la siguiente estructura:

```
Id:         1
Winner:     236863
Target:     00000000
Solution:   08916551 (validated)
Votes:      1/1
Wallets:    236863:1

Id:         2
Winner:     236863
Target:     08916551
Solution:   05970796 (validated)
Votes:      1/1
Wallets:    236863:2
```

Figura 5: Estructura esperada para el registro del primer minero

Además, podemos verificar que el registrador se genera correctamente para cada minero, de forma que cada minero toma registro de sus candidaturas a ganar la ronda:

```
./mrush 3 8 & ./mrush 5 8 & ./mrush 10 8 & ./mrush 10 1 & wait
[21:38:32] Miner 122611 added to system

===== ACTIVE MINERS =====
- Process  122611

[21:38:32] Miner 122612 added to system

===== ACTIVE MINERS =====
- Process  122611
- Process  122612

[21:38:32] Miner 122613 added to system

===== ACTIVE MINERS =====
- Process  122611
- Process  122612
- Process  122613

[21:38:32] Miner 122614 added to system

===== ACTIVE MINERS =====
- Process  122611
- Process  122612
- Process  122613
- Process  122614

Starting mining!

Winner 122613 => [ Y Y ] => Accepted
Winner 122611 => [ N Y Y ] => Accepted
Winner 122611 => [ Y Y Y ] => Accepted
Winner 122612 => [ Y Y Y ] => Accepted
Winner 122613 => [ Y Y Y ] => Accepted
Winner 122613 => [ Y Y Y ] => Accepted
Winner 122611 => [ Y Y Y ] => Accepted
Winner 122613 => [ Y Y Y ] => Accepted
Winner 122613 => [ Y Y Y ] => Accepted
Winner 122612 => [ Y Y Y ] => Accepted
Winner 122612 => [ Y Y Y ] => Accepted
Winner 122612 => [ Y Y Y ] => Accepted
Winner 122612 => [ Y Y Y ] => Accepted
Winner 122613 => [ Y Y Y ] => Accepted
Winner 122613 => [ Y Y Y ] => Accepted
Winner 122613 => [ Y Y Y ] => Accepted
Winner 122612 => [ Y Y Y ] => Accepted
```

```
0 1 zsh
danyyil at debian13 in ~/INFORMATICA/2º/SOPER/P2/Miner
.:: head -13 122613.log
Id:         1
Winner:     122613
Target:     00000000
Solution:   08916551 (validated)
Votes:      2/2
Wallets:    122613:1

Id:         5
Winner:     122613
Target:     01835314
Solution:   09331340 (validated)
Votes:      3/3
Wallets:    122613:2
danyyil at debian13 in ~/INFORMATICA/2º/SOPER/P2/Miner
.:: head -13 122611.log
Id:         2
Winner:     122611
Target:     08916551
Solution:   05970796 (validated)
Votes:      2/3
Wallets:    122611:1

Id:         3
Winner:     122611
Target:     05970796
Solution:   00937352 (validated)
Votes:      3/3
Wallets:    122611:2
danyyil at debian13 in ~/INFORMATICA/2º/SOPER/P2/Miner
.:: head -13 122612.log
Id:         4
Winner:     122612
Target:     00937352
Solution:   01835314 (validated)
Votes:      3/3
Wallets:    122612:1

Id:         10
Winner:     122612
Target:     08021305
Solution:   04673876 (validated)
Votes:      3/3
Wallets:    122612:2
```

Figura 6: Comprobación de que cada logger registra única y correctamente a su proceso padre (minero)

En las líneas resaltadas de blanco, vemos que el ganador de la primera ronda es 122613, como consta en su registro, al igual que la segunda y tercera ronda, quedando constancia en su registro e igual para el resto. Además, viendo los registros verificamos que gana un solo minero por ronda.

3.1.5. Rechazo de soluciones incorrectas

Para verificar el rechazo de soluciones incorrectas, hemos establecido una condición de definición de una macro "FAKE"que, en caso de que se defina a la hora de compilar, habrá un 10 % de probabilidad de que el proceso genere una solución falsa:

```c
/* Cada minero pone una semilla aleatoria basada en el pid */
#ifdef FAKE
  srand(getpid() ^ time(NULL));
#endif /* ifdef FAKE */

#ifdef FAKE
        if (rand() % 100 < 10) {
          sol = 99999999; // Ponemos una solucion false con 0,1 de probabilidad
        }
#endif /* ifdef FAKE */
```

Además, se ha implementado una regla en el `Makefile` para facilitar la ejecución

```
fake: CFLAGS += -DFAKE
fake:
  $(MAKE) clean
  $(MAKE) all CFLAGS="$(CFLAGS)"

run_fake: fake
  ./$(EXE) 3 8 & ./$(EXE) 5 8 & ./$(EXE) 10 8 & ./$(EXE) 10 1 & wait
```

De esta manera, obtenemos el siguiente output:

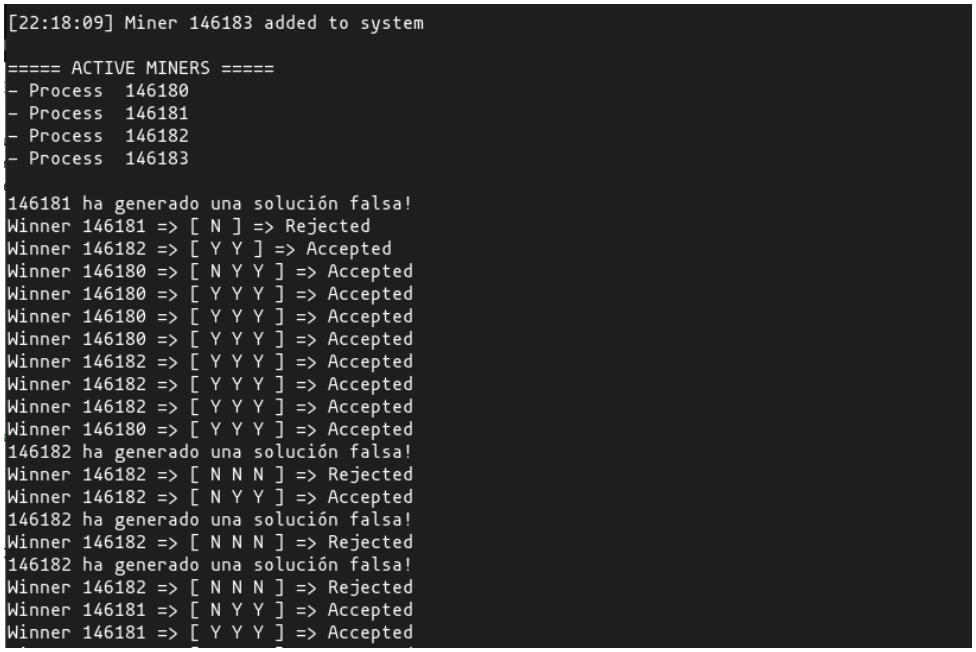


Figura 7: Output al generar soluciones inválidas

3.2. Parámetros de entrada generales

Se ha implementado un control exhaustivo de errores tanto en la inicialización de recursos del sistema como en el uso de las distintas funciones auxiliares. En particular, se verifica la correcta apertura de semáforos, tuberías y ficheros compartidos, así como el resultado de las operaciones sensibles del sistema, como `fork()`, `sigaction()`, `sem_open()`, `pipe()`, `pthread_create()` o `timer_create()`.

En las funciones auxiliares se emplea una validación estricta de argumentos, combinando comprobaciones explícitas y macros de salida por error. Esto permite detectar de forma temprana estados inválidos del sistema y simplifica la localización de fallos durante la depuración.

```
#define die(msg)                                                    \
  do {                                                              \
    perror(msg);                                                    \
    exit(EXIT_FAILURE);                                             \
  } while (0)
```

```c
// Ejemplo de uso
if (pipe(miner_pipe) == -1)
  die("pipe");
```

Asimismo, se han comprobado manualmente situaciones excepcionales como:

- Fallo en la creación de hilos.
- Fallo en la apertura de semáforos nombrados.
- Recepción de señales no esperadas.
- Terminación abrupta de un minero durante una ronda.

Como se puede ver en la siguiente imagen, en el caso de un fallo en la creación de hilos, la ejecución se corta y el programa termina de manera prematura:

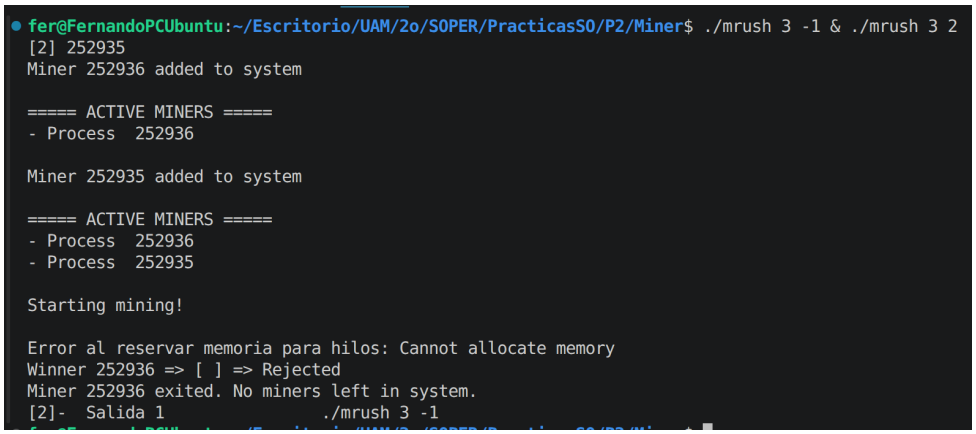


Figura 8: Al pasar un número de hilos negativo, la ejecución se corta

3.3. Parámetros del programa principal

Se ha realizado un control exhaustivo de los parámetros de entrada del programa principal. En esta práctica, el ejecutable se invoca como:

```
./mrush <N_SECS> <N_THREADS>
```

donde `N_SECS` representa el tiempo máximo de vida del proceso minero y `N_THREADS` el número de hilos de minería internos. Ambos parámetros deben ser enteros positivos y coherentes con el funcionamiento esperado del sistema.

Este control es especialmente importante porque un valor inválido en el número de hilos podría provocar comportamientos no deseados, desde un uso desproporcionado de recursos hasta intentos de creación masiva de hilos. Por ello, se verifica que los argumentos tengan el formato adecuado y que se encuentren dentro de rangos razonables antes de iniciar la ejecución del programa.

Para ello, hemos creado la siguiente función de parsing seguro:

```c
u64 str_to_u64(char *input) {
  char *endptr;
  /* No usamos atoi porque los numeros usados son my grandes, es posible que
   * haya overflow */
  u64 out = strtoull(input, &endptr, 10);
  /* En el manual dice que en caso de desbordamiento strtoul anota ERANGE en
   * errno. Si los datos se guardaran en tamannos menores a u64, los negativos
    desbordarian */
  if (errno == ERANGE)
    die("Desbordamiento! El numero introducido es demasiado grande o negativo");
  if (strchr(input, '-'))
    die_msg("No se aceptan parametros negativos");
  /* strtoul coloca endptr en el primer digito que no sea un numero luego si
   * coincide en el principio es que no habia ningun numero valido */
  if (endptr == input)
    die("Parametro invalido, debe ser solo un numero");
  if (*endptr != '\0')
    die("Parametro invalido, debe ser solo un numero");
  return out;
}
```

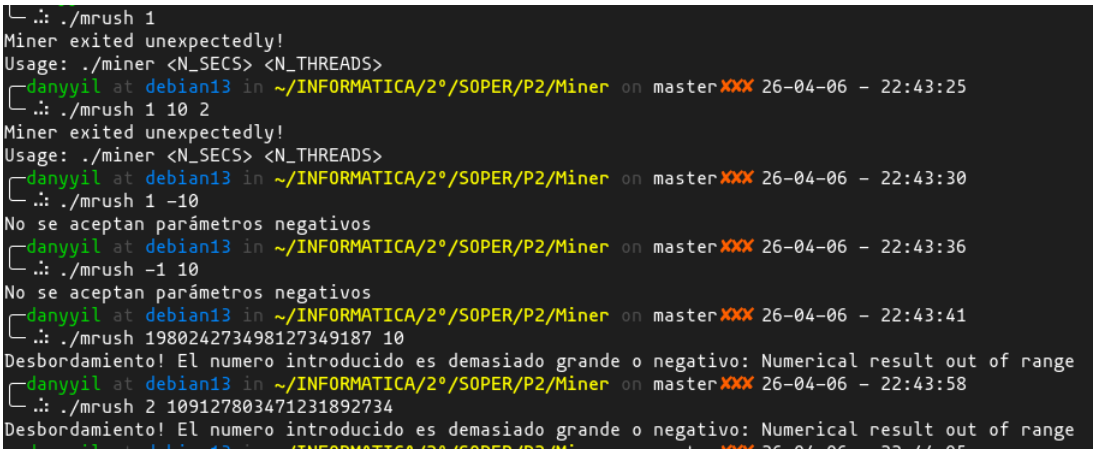


Figura 9: Resultados al introducir parámetros inválidos en la práctica 2

3.4. Pruebas Unitarias

Se ha diseñado un *framework* de pruebas unitarias y de integración automatizado. A diferencia de la primera práctica, donde las pruebas se centraban en computar el PoW, el foco de estos test es la concurrencia, la sincronización, la comunicación interprocesos y la tolerancia a fallos.

Las pruebas evalúan tanto el funcionamiento normal como situaciones límite para garantizar que el sistema no sufra interbloqueos:

- **Sincronización y Ficheros:** Se ha validado la correcta creación y destrucción de los semáforos. Además, se simularon distintos escenarios de votación (mayorías y rechazos) para verificar que el algoritmo de consenso funciona sin quedarse atascado.
- **Comunicación entre procesos:** Usando la función `fork()` dentro de los tests, se simuló la interacción entre el minero y el registrador. Se comprobó que los datos viajan correctamente por las tuberías (*pipes*) y se escriben bien en el archivo de registro.
- **Señales y Esperas:** Se inyectaron señales externas (`SIGUSR1`, `SIGUSR2`) y eventos de límite de tiempo (*timeouts*). Con esto demostramos que los mineros interrumpen su trabajo al instante cuando se les pide, y que las funciones de espera no se bloquean de forma infinita si fallan las condiciones.
- **Validación de Memoria (Valgrind):** Toda la batería de pruebas finalizó con **cero fugas de memoria**. Esto certifica no solo el buen uso de `malloc` y `free`, sino también que todos los procesos cierran correctamente sus archivos y semáforos antes de terminar, incluso en caso de error.

```
Ejecutando TODOS los tests...                    23:48:35 [15/569]
Entrando en el directorio: files_test
Compilando en files_test...
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g files_test.c
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g files.c
gcc -o files_test files_test.o files.o -pthread
Ejecutando files_test en modo normal...
Running all test for module Files/Consensus:
files_test.c line 66 test1_open_pipes: PASS
files_test.c line 81 test2_close_pipes: PASS
files_test.c line 89 test3_mutex_init: PASS
files_test.c line 105 test4_pids_io: PASS
Winner 206875 => [ Y Y N ] => Accepted
files_test.c line 119 test5_voting_system: PASS
Tests passed 100%
Limpiando files_test...
Entrando en el directorio: logger_test
Compilando en logger_test...
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g logger_test.c
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g logger.c
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g files.c
gcc -o logger_test logger_test.o logger.o files.o -pthread
Ejecutando logger_test en modo normal...
Running Logger tests...
logger_test.c line 59 test1_logger_ipc: PASS
logger_test.c line 62 test1_logger_ipc: PASS
Tests passed 100%
Limpiando logger_test...
Entrando en el directorio: miner_test
```

```
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g files.c
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g miner.c
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g logger.c
gcc -c -Wall -Wextra -std=gnu99 -pedantic -g pow.c
gcc -o miner_test miner_test.o files.o miner.o logger.o pow.o
Ejecutando miner_test en modo normal...
Running all test for module Miner (Sync & Concurrency):
miner_test.c line 82 test1_calcular_solucion: PASS
miner_test.c line 97 test1_wait_votes_success: PASS
miner_test.c line 116 test2_wait_votes_timeout: PASS
miner_test.c line 145 test1_wait_more_miners_success: PASS
miner_test.c line 171 test2_wait_more_miners_timeout: PASS
Tests passed 100%
Limpiando miner_test...
Entrando en el directorio: pow_test
Compilando en pow_test...
cc -c -Wall -Wextra -std=gnu99 -pedantic -g pow_test.c
pow_test.c: In function 'test3_pow':
pow_test.c:71:57: warning: integer constant is so large that it is unsigned
   71 |   PRINT_TEST_RESULT(fx == pow_hash(18446744073709551615));
      |                                                         ^
cc -c -Wall -Wextra -std=gnu99 -pedantic -g pow.c
cc -o pow_test *.o
Ejecutando pow_test en modo normal...
Running all test for module Pow:
pow_test.c line 59 test1_pow: PASS
pow_test.c line 65 test2_pow: PASS
pow_test.c line 71 test3_pow: PASS
Tests passed 100%
```

Figura 10: Traza de ejecución de pruebas unitarias y funcionales

3.5. Pruebas de memoria

Además de la validación funcional, se han realizado pruebas de memoria mediante `valgrind`, tanto sobre ejecuciones individuales del programa como sobre escenarios representativos con varios mineros activos. El objetivo ha sido detectar posibles fugas de memoria, accesos inválidos y recursos no liberados correctamente.

Estas pruebas son especialmente relevantes en esta práctica debido a la combinación de:

- Procesos creados con `fork()`
- Hilos POSIX
- Semáforos nombrados
- Temporizadores
- Estructuras dinámicas compartidas entre distintas partes del sistema.

En el `Makefile` se ha incluido una regla específica para facilitar la ejecución de `valgrind` sobre el programa principal, redirigiendo el resultado a un fichero de log para su posterior análisis.

```
# Ejemplo de uso
$ valgrind --leak-check=full --show-leak-kinds=all \
  --track-origins=yes --log-file=memcheck.log ./mrush 10 4
```

Para verificar la ausencia de fugas, la regla en `Makefile` realiza la detección sobre 4 mineros iniciados simultáneamente:

```
VALG=valgrind --leak-check=full --show-leak-kinds=all \
     --track-origins=yes --trace-children=yes --log-file=memcheck-%p.log

valgrind: all
  $(VALG) ./$(EXE) 3 8 & $(VALG) ./$(EXE) 5 8 & $(VALG) ./$(EXE) 10 8 & \
  $(VALG) ./$(EXE) 10 4 & wait
```

```
# RESULTADO MINERO 166092
==166093== Memcheck, a memory error detector
==166093== Copyright (C) 2002-2024, and GNU GPLd, by Julian Seward et al.
==166093== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright info
==166093== Command: ./mrush 3 8
==166093== Parent PID: 166092
==166093==
==166093==
==166093== HEAP SUMMARY:
==166093==     in use at exit: 0 bytes in 0 blocks
==166093==   total heap usage: 108 allocs, 108 frees, 139,094 bytes allocated
==166093==
==166093== All heap blocks were freed -- no leaks are possible
==166093==
==166093== For lists of detected and suppressed errors, rerun with: -s
==166093== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
# RESULTADO REGISTRADOR ASOCIADO 166093
==166100== Memcheck, a memory error detector
==166100== Copyright (C) 2002-2024, and GNU GPLd, by Julian Seward et al.
==166100== Using Valgrind-3.24.0 and LibVEX; rerun with -h for copyright info
==166100== Command: ./mrush 3 8
==166100== Parent PID: 166093
==166100==
==166100==
==166100== HEAP SUMMARY:
==166100==     in use at exit: 0 bytes in 0 blocks
==166100==   total heap usage: 8 allocs, 8 frees, 292 bytes allocated
==166100==
==166100== All heap blocks were freed -- no leaks are possible
==166100==
==166100== For lists of detected and suppressed errors, rerun with: -s
==166100== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Los resultados obtenidos indican que no existen fugas de memoria relevantes en la ejecución normal del sistema, lo cual confirma una correcta liberación de recursos dinámicos tanto en el proceso minero como en el registrador.

3.6. Pruebas de sincronización y concurrencia

Dado que esta segunda práctica se centra en la coordinación entre múltiples procesos mediante señales y semáforos, se han realizado pruebas específicas de sincronización para comprobar la robustez del sistema frente a condiciones de carrera y pérdida de señales.

En particular, se ha verificado que:

- Las señales `SIGUSR1` y `SIGUSR2` no se pierden durante la transición entre estados,
  Al lanzar varios mineros a la vez, todos empiezan a minar a la vez (Mensaje "Start Mining!), y todos entran en votación (tantos Y o N hay como procesos votantes).

```
===== ACTIVE MINERS =====
- Process  261952
- Process  261953

Starting mining!

Miner 261954 added to system

===== ACTIVE MINERS =====
- Process  261952
- Process  261953
- Process  261954

Winner 261952 => [ Y ] => Accepted
Winner 261953 => [ Y Y ] => Accepted
```

Figura 11: Las señales no se pierden

  Cabe destacar que para el primer resultado sólo tenemos un votante, porque el tercer minero no participa en la primera ronda.

- La espera se realiza mediante `sigsuspend()`, tal y como recomienda el enunciado
- El semáforo asociado al proceso ganador garantiza que solo exista un ganador por ronda
- Los accesos a los ficheros compartidos quedan protegidos por semáforos mutex.
  Se comprueba gracias al fichero de registro, limpio y sin corrupciones.

```
Id:         1
Winner:     236863
Target:     00000000
Solution:   08916551 (validated)
Votes:      1/1
Wallets:    236863:1

Id:         2
Winner:     236863
Target:     08916551
Solution:   05970796 (validated)
Votes:      1/1
Wallets:    236863:2
```

Figura 12: Registro limpio

Gracias al registro también podemos comprobar que sólo existe un ganador por ronda.

- Un minero que termina su vida útil durante la ejecución no bloquea permanentemente al resto del sistema.
  Se comprueba mediante la ejecución de la siguiente instrucción en terminal:

```
$ ./mrush 10 4 & ./mrush 10 4 & ./mrush 5 4
```

Además, se ha utilizado `helgrind` de `valgrind` para inspeccionar posibles problemas de concurrencia en la parte multihilo del cálculo de la POW. Aunque el sistema global es multiproceso y buena parte de la sincronización interproceso se resuelve con semáforos nombrados, sigue siendo necesario comprobar el correcto acceso a variables compartidas entre los hilos del minero.

```
$ valgrind --tool=helgrind --log-file=thread_check.log ./mrush 10 4
```

Los resultados de estas pruebas han permitido confirmar que la estructura de sincronización elegida es adecuada para esta práctica y que el uso combinado de señales bloqueadas, `sigsuspend()` y semáforos evita la mayor parte de los riesgos clásicos de concurrencia en este tipo de sistemas. No obstante, al igual que en la primera práctica, no hemos sabido como prevenir el acceso concurrente por parte de los hilos, por lo que helgrind seguirá dando error.

3.7. Pruebas de rendimiento

En este apartado se evalúa el comportamiento del sistema distribuido bajo diferentes niveles de carga, con el objetivo de identificar el equilibrio óptimo entre el paralelismo interno de los hilos y la contención generada por la comunicación entre procesos.

Esta práctica introduce una sobrecarga operativa derivada de los semáforos, el envío de señales en tiempo real y el acceso concurrente a ficheros compartidos. Mediante el uso de la herramienta `time`, se analizan métricas críticas como el User Time y el System Time (coste de sincronización) para certificar la escalabilidad del diseño propuesto ante un incremento en el número de mineros.

3.7.1. Redes heterogéneas

El objetivo de esta prueba es demostrar que el minero con más hilos tiende a ganar más monedas. Para ello, en el makefile de tests (tests.mk), llamamos al comando make test_hetero, y obtenemos los siguientes datos.

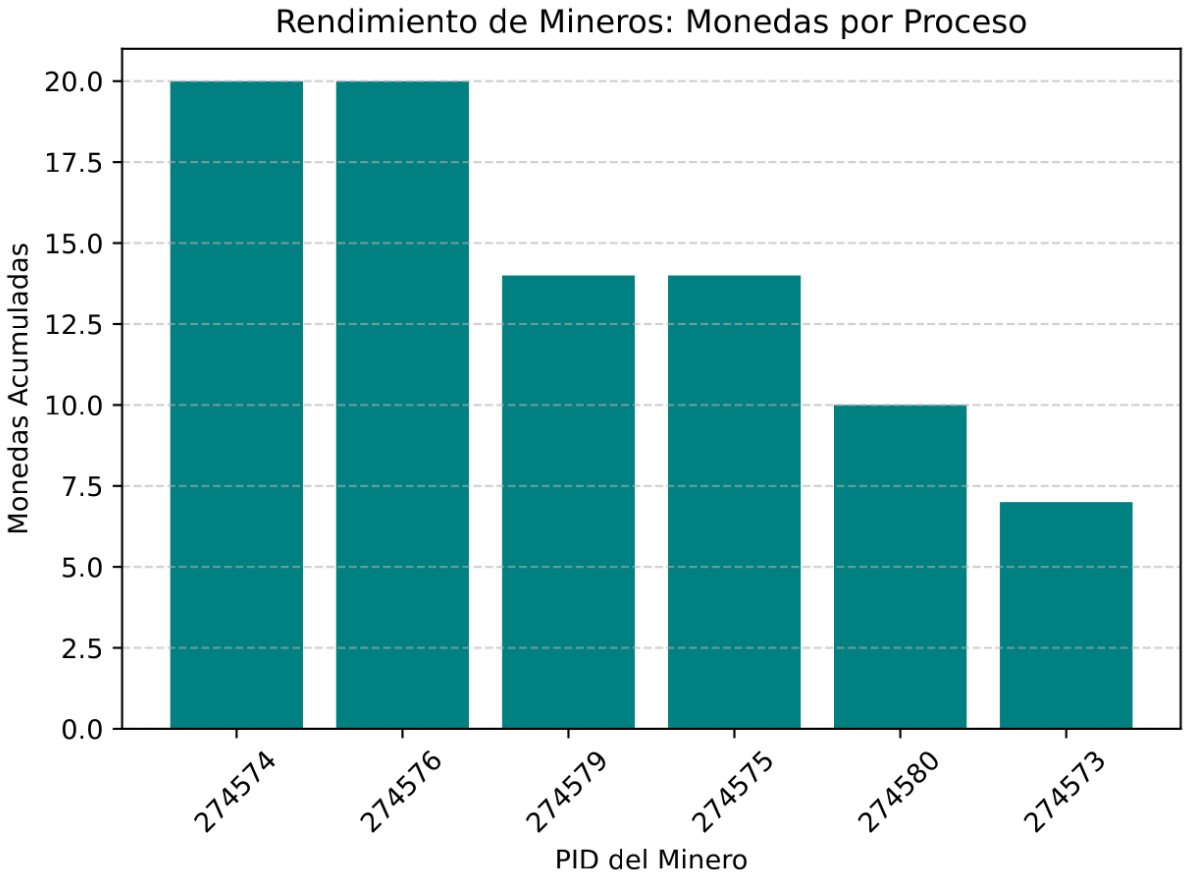


Figura 13: Resultados monedas en función de hilos

En el resultado, los mineros con más monedas fueron aquellos que se ejecutaron con 14 y 10 hilos, mientras que el resto obtenían menos, tanto por tener menos hilos como por tener demasiados.

3.7.2. Sobrecarga de IPC

Por otro lado, hemos realizado la prueba de tener numerosos mineros en el sistema. Al tener muchos, esperamos que haya mucha colisión de semáforos, causando que aumente el tiempo de sistema operativo.

```
=====================================================
 PRUEBA 3: Estrés de Red (IPC Overhead)
 Fijos: 5s de vida | 4 HILOS por minero
=====================================================

make[1]: Entering directory '/home/danyyil/INFORMATICA/2º/SOPER/P2/Miner'
make[1]: Leaving directory '/home/danyyil/INFORMATICA/2º/SOPER/P2/Miner'
---> Red de 2 MINEROS simultáneos
( ../mrush 5 4 & ../mrush 5 4 & wait; )  1.00s user 0.09s system 18% cpu 6.008 total

---> Red de 3 MINEROS simultáneos
make[1]: Entering directory '/home/danyyil/INFORMATICA/2º/SOPER/P2/Miner'
make[1]: Leaving directory '/home/danyyil/INFORMATICA/2º/SOPER/P2/Miner'
( ../mrush 5 4 & ../mrush 5 4 & ../mrush 5 4 & wait; )  1.84s user 0.15s system 33% cpu 6.009 total

---> Red de 5 MINEROS simultáneos
make[1]: Entering directory '/home/danyyil/INFORMATICA/2º/SOPER/P2/Miner'
make[1]: Leaving directory '/home/danyyil/INFORMATICA/2º/SOPER/P2/Miner'
( ../mrush 5 4 & ../mrush 5 4 & ../mrush 5 4 & ../mrush 5 4 & ../mrush 5 4 & )  2.84s user 0.11s system 49% cpu 6.009 total

---> Red de 10 MINEROS simultáneos (Colisión de semáforos)
make[1]: Entering directory '/home/danyyil/INFORMATICA/2º/SOPER/P2/Miner'
make[1]: Leaving directory '/home/danyyil/INFORMATICA/2º/SOPER/P2/Miner'
( ../mrush 5 4 & ../mrush 5 4 & ../mrush 5 4 & ../mrush 5 4 & ../mrush 5 4 & )  5.76s user 0.28s system 100% cpu 6.010 total

---> Red de 20 MINEROS simultáneos (Alta colisión de semáforos)
make[1]: Entering directory '/home/danyyil/INFORMATICA/2º/SOPER/P2/Miner'
make[1]: Leaving directory '/home/danyyil/INFORMATICA/2º/SOPER/P2/Miner'
( ../mrush 5 4 & ../mrush 5 4 & ../mrush 5 4 & ../mrush 5 4 & ../mrush 5 4 & )  6.07s user 0.34s system 106% cpu 6.009 total
```

Figura 14: Aumento del tiempo de sistema al aumentar cantidad de mineros

3.8. Conclusiones de las pruebas

A modo de síntesis, las pruebas realizadas permiten concluir que el sistema implementado cumple los objetivos funcionales de la práctica y presenta un comportamiento coherente desde el punto de vista de concurrencia.

Las pruebas funcionales validan el correcto funcionamiento de la red de mineros, la coordinación mediante señales, la exclusión mutua con semáforos y el registro de resultados por parte del logger.

4. FUNCIONALIDADES ADICIONALES

Más allá del estricto cumplimiento de los requisitos establecidos en la descripción de la práctica, el proyecto incorpora una serie de mejoras orientadas a la optimización, seguridad y automatización del sistema distribuido:

- Simulación de mineros maliciosos: Para testear de forma rigurosa la robustez del sistema de votaciones, se ha añadido una regla de compilación especial (`make fake`) en el `Makefile` que inyecta la macro `-DFAKE`. Esta implementación altera el comportamiento de la función ganadora, induciendo soluciones falsas con un 10 % de probabilidad. Esto certifica que la red rechaza los bloques erróneos y revierte el target adecuadamente.
- Optimización de Memoria y Padding de Estructuras: Se ha aplicado un diseño consciente del hardware en la arquitectura de datos. En las estructuras, los campos se han ordenado estratégicamente (dejando variables de menor tamaño al final) para minimizar el padding generado por el compilador, logrando alinear las estructuras a múltiplos de 8 bytes, que es la palabra mínima. Esto reduce el ancho de banda necesario en las comunicaciones por tuberías entre el minero y el registrador.
- Testing y Sanitización Integrada: El `Makefile` no actúa únicamente como ensamblador, sino como un entorno de integración. Se han programado reglas (`test_hetero`, `test_scaling`, `test_network`) que ejecutan perfiles de sobrecarga de forma aislada, limpiando los ficheros compartidos entre rondas para evitar contaminación de PIDs. Adicionalmente, incluye directivas para invocar `Valgrind` y `Helgrind`, asegurando la ausencia total de fugas de memoria y condiciones de carrera entre los procesos (no hilos).

5. CUESTIONES

5.1. ¿Se produce algún problema de concurrencia en el sistema descrito?

Sí, el sistema presenta múltiples problemas de concurrencia críticos debido a la naturaleza distribuida y paralela de la red:

- Condiciones de Carrera: Múltiples mineros compiten simultáneamente por ser el primero en reclamar la victoria tras encontrar una solución al Proof of Work. Si dos mineros encuentran la solución en el mismo milisegundo, podrían intentar proclamarse ganadores a la vez.
- Acceso concurrente a ficheros (Secciones Críticas): Todos los procesos necesitan leer y escribir en recursos compartidos de forma constante: unirse o salir de la red (PIDs.pid), leer el objetivo o escribir la nueva solución (Target.tgt), y registrar sus votos simultáneamente (Voting.vot). Sin control, las escrituras se solaparían, corrompiendo los datos.
- Pérdida de señales: Las señales asíncronas (SIGUSR1 y SIGUSR2) pueden llegar cuando un proceso está ocupado o en mitad de la gestión de otra señal, lo que podría provocar bloqueos si un minero se queda esperando un aviso que "se ha perdido".

5.2. ¿Cómo se ha solucionado en el planteamiento dado?

Hemos resuelto estos problemas aplicando primitivas de sincronización del sistema operativo:

- Semáforos (Mutex): Se han creado 4 semáforos con valor inicial 1 (PID_MUTEX, TARGET_MUTEX, VOTES_MUTEX y WINNER_MUTEX) que garantizan la exclusión mutua. Ningún proceso puede leer o escribir en un fichero sin haber hecho un sem_wait previo.
- Control del winner: Para resolver la condición de carrera del ganador, se utiliza sem_trywait(sems->win). El primer minero que lo intenta, entra y bloquea el semáforo. Si una fracción de segundo después otro minero intenta lo mismo, sem_trywait fallará, asumiendo su rol como votante.
- Suspensión segura: Para evitar la pérdida de señales, se ha implementado el uso combinado de sigprocmask y sigsuspend. Esto permite que el proceso bloquee temporalmente las señales mientras realiza tareas críticas y las desbloquee únicamente en el momento exacto de suspenderse, garantizando que ninguna señal se procese a destiempo.

5.3. ¿Es sencillo, o siquiera factible, organizar un sistema de este tipo usando únicamente señales?

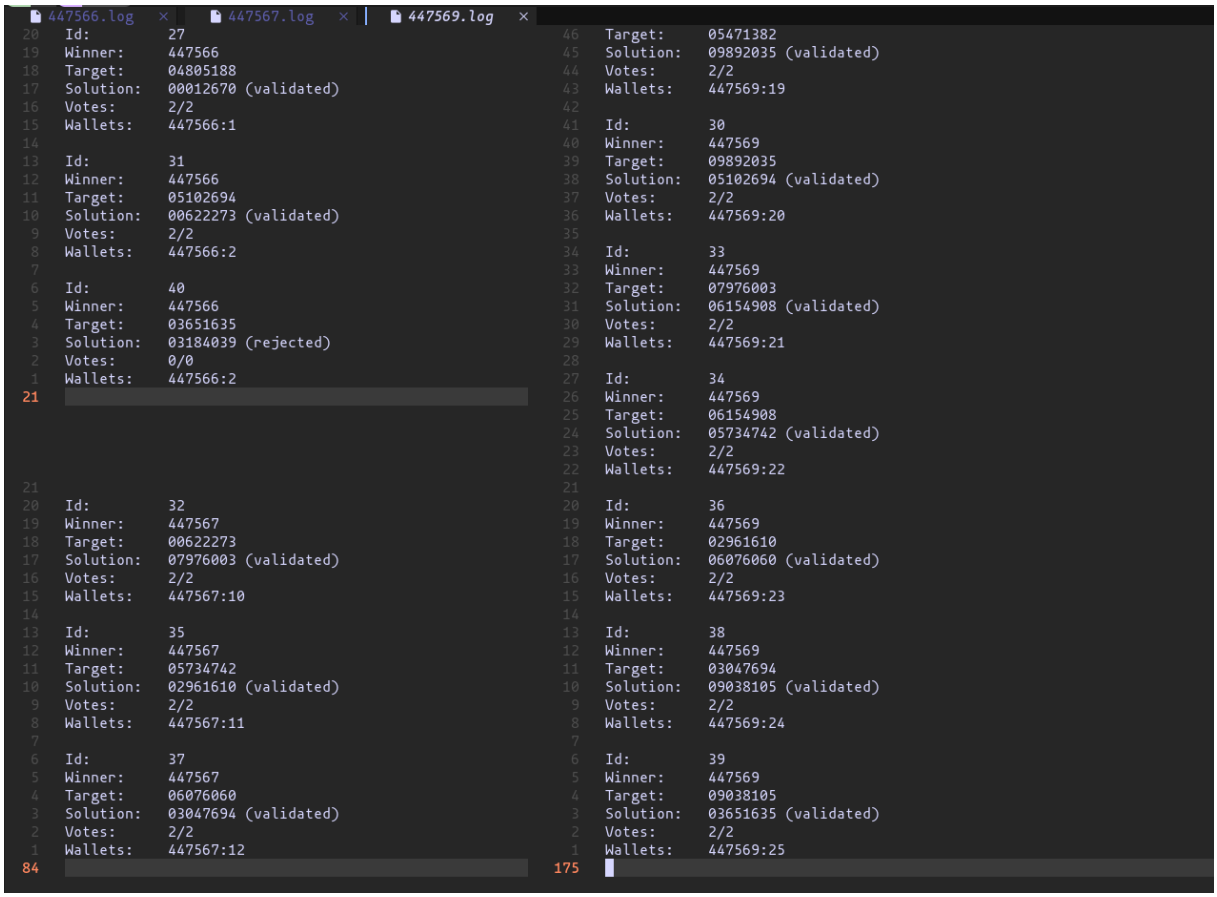
En teoría, sería factible pero nada sencillo. Por un lado, la gestión del ganador sería más delicada. Ahora se gestiona con un semáforo win, que una vez tomado no se desbloquea hasta que terminan los votos. Si se hiciera con señales, el ganador tendría que hacer un broadcast al resto de procesos, por lo que si llegan dos prácticamente a la vez no sería tan fácil como con el semáforo (debido a que estos operan a nivel de kernel), haciendo que tuviéramos que recrear a nivel usuario mecanismos de kernel. Además, el encolamiento de señales durante la suspensión de los votantes y el bloqueo pasivo requeriría un sistema muy complejo y propenso a interbloqueos. El diseño con semáforos, en nuestra opinión, es óptimo.

5.4. ¿Cuáles son los mineros que tienden a ganar más monedas?

Los mineros que tienden a ganar más monedas son aquellos lanzados con un mayor número de hilos. Al paralelizar el cálculo interno, estos mineros pueden evaluar múltiples combinaciones del hash por segundo dividiendo el espacio de búsqueda.

Aunque el Proof of Work tiene un componente estocástico y un minero de 1 hilo podría ganar una ronda por suerte, a lo largo del tiempo la estadística favorece a la fuerza computacional de los hilos.

Una prueba fácil sería la ejecución de 3 mineros, uno con 1 hilo, otro con 4 y otro con 16:



```
447566.log
Id:         27
Winner:     447566
Target:     04805188
Solution:   00012670 (validated)
Votes:      2/2
Wallets:    447566:1

Id:         31
Winner:     447566
Target:     05102694
Solution:   00622273 (validated)
Votes:      2/2
Wallets:    447566:2

Id:         40
Winner:     447566
Target:     03651635
Solution:   03184039 (rejected)
Votes:      0/0
Wallets:    447566:2
```

```
447567.log
Id:         32
Winner:     447567
Target:     00622273
Solution:   07976003 (validated)
Votes:      2/2
Wallets:    447567:10

Id:         35
Winner:     447567
Target:     05734742
Solution:   02961610 (validated)
Votes:      2/2
Wallets:    447567:11

Id:         37
Winner:     447567
Target:     06076060
Solution:   03047694 (validated)
Votes:      2/2
Wallets:    447567:12
```

```
447569.log
Target:     05471382
Solution:   09892035 (validated)
Votes:      2/2
Wallets:    447569:19

Id:         30
Winner:     447569
Target:     09892035
Solution:   05102694 (validated)
Votes:      2/2
Wallets:    447569:20

Id:         33
Winner:     447569
Target:     07976003
Solution:   06154908 (validated)
Votes:      2/2
Wallets:    447569:21

Id:         34
Winner:     447569
Target:     06154908
Solution:   05734742 (validated)
Votes:      2/2
Wallets:    447569:22

Id:         36
Winner:     447569
Target:     02961610
Solution:   06076060 (validated)
Votes:      2/2
Wallets:    447569:23

Id:         38
Winner:     447569
Target:     03047694
Solution:   09038105 (validated)
Votes:      2/2
Wallets:    447569:24

Id:         39
Winner:     447569
Target:     09038105
Solution:   03651635 (validated)
Votes:      2/2
Wallets:    447569:25
```

Figura 15: Resultado de los registros para mineros con 1, 4 y 16 hilos

Otra prueba es la dada por la figura 13

6. CONCLUSIONES

Esta práctica nos ha ayudado a entender mucho mejor cómo funcionan los sistemas donde varias tareas se ejecutan al mismo tiempo. Al programar esta red de mineros, hemos dejado atrás la programación paso a paso para enfrentarnos a los retos de ejecutar y coordinar procesos en paralelo.

Lo más importante ha sido la comunicación y sincronización. Hemos aprendido a usar semáforos para que los mineros no escriban a la vez en los ficheros compartidos, tuberías para enviar datos al registrador sin dejar de trabajar, y señales para avisarse entre ellos de forma rápida.

El mayor obstáculo que tuvimos fue controlar cuando muchos eventos ocurren a la vez. Para evitar que dos mineros intentaran ganar en el mismo milisegundo o que se perdieran avisos importantes, tuvimos que usar funciones del sistema como `sigprocmask` y `sigsuspend`. Además, usar herramientas como `Valgrind` fue imprescindible para encontrar y arreglar errores de memoria que a simple vista no se veían.

En resumen, hemos conseguido crear un sistema estable y funcional. Además, al añadir pruebas automáticas, hemos podido comprobar de forma muy visual cómo las decisiones que tomamos al escribir el código (como el número de hilos) afectan directamente a la velocidad y al uso del procesador.

7. ANEXO

Proporcionamos la documentación generada en pdf por Doxygen, pero siempre se puede generar con el comando ”make doc”.

Los tests unitarios se han añadido a una carpeta separada para no interferir con el programa principal. Se pueden ejecutar fácilmente con el script de bash añadido. La funcionalidad es la siguiente:

```
./script-test <0|1> "Nombre_test"
```

El flag 0 indica que no se ejecute el test con valgrind, al contrario que el 1. El nombre de test especifica cuál de los tests ejecutar. Si no se especifica el nombre ejecutará todos.

A lo largo de la práctica hemos realizado ciertas decisiones de diseño, que explicamos y exploramos otras alternativas

7.1. Comienzo de temporizador al minar

La forma en la que hemos decidido realizar el sistema, es que el temporizador no comienza hasta que no comienza la minería. Hemos pensado que esto tiene más sentido frente a que comience inmediatamente, principalmente porque si el primer minero tiene que esperar digamos a la aparición del segundo, si la duración es muy reducida, podría terminar antes de que el segundo apareciese, y nuevamente el segundo se quedaría solo y terminaría sin que se realice ninguna minería. De esta forma, al iniciar un solo minero, el programa queda en ”stall” o en espera de la adición de un segundo.

Si se quisiera cambiar el sistema de forma que el temporizador comenzara en cuando el minero se incorpora al sistema, se deben realizar los siguientes cambios en el fichero `miner.c`

```c
//Linea 232
  wait_more_miners(sems);

  if (first_miner) {
    printf("Starting mining!\n\n");
  }

  timer_t m_timer;
  miner_set_alarm(args->time, &m_timer);
```

Y se movería el inicio de la alarma al principio de la función:

```c
void minero(Miner_data *args, i32 *miner_pipe, i32 *logger_pipe,
            Miner_Mutexes *sems) {
  assert(args != NULL);
  assert(miner_pipe != NULL);
  assert(logger_pipe != NULL);
  assert(sems != NULL);

  setup_signals();
  timer_t m_timer;
  miner_set_alarm(args->time, &m_timer);
```

Por otro lado, cuando múltiples mineros encuentran la solución casi al mismo tiempo, todos intentan acceder al semáforo del ganador (`WINNER_MUTEX`). En lugar de usar la función bloqueante estándar

`sem_wait`, se ha decidido utilizar `sem_trywait`.

Si usáramos `sem_wait`, el primer minero entraría a la sección crítica y el resto se quedaría bloqueado en la cola del semáforo. Al usar `sem_trywait`, el primero consigue acceder, pero a los demás la función les devuelve un error instantáneo. Esto permite que los mineros perdedores se den cuenta inmediatamente de que alguien se les ha adelantado, abandonen su intento de ser ganadores y pasen automáticamente a su rol de votantes sin perder tiempo ni quedarse bloqueados.

Además, la función `exit_network` incluye una lógica de limpieza mediante la cual el minero comprueba cuántos procesos activos quedan en el fichero antes de salir. Si detecta que es el último, invoca las funciones `unlink` y `sem_unlink`.

Si los ficheros (`Target.tgt`, `PIDs.pid`, etc.) y los semáforos no se destruyeran, una ejecución futura del programa podría leer PIDs de procesos que ya no existen, o encontrarse un semáforo bloqueado de una sesión anterior que terminó mal. Esta decisión hace que el programa sea resistente y no requiera que el usuario borre archivos a mano entre pruebas.